نوبتین — راهنمای انتشار روی گیت‌هاب

مرحله‌به‌مرحله، برای کسی که تا حالا مخزن نکرده است. از ساخت حساب تا لینکی که کارفرما باز می‌کند و نظر می‌دهد.

۱۰ بخش | حدود ۴۰ تا ۶۰ دقیقه | نسخهٔ ۱.۰ | ۱۱ آبان ۱۴۰۵

۹۷	۳٫۳ مگابایت	۳۵ بخش	۰
فایل آمادهٔ انتشار (بدون دیتابیس و کش)	حجم کل، که ۱٫۹ مگ آن اسکرین‌شات است	آزمون پایان‌به‌پایان + ۳ شغل CI	رمز، شمارهٔ واقعی یا دیتابیس در مخزن

این راهنما چه چیزی را پوشش می‌دهد

1. حساب گیت‌هاب و تأیید دومرحله‌ای (۲FA)
2. ساخت مخزن (Repository) به نام `nobatin`
3. ساخت کلید دسترسی (Token) — فقط برای فرستادن فایل‌ها
4. اجرای دستورهای `git` روی همین ماشین (یا راه گرافیکی با GitHub Desktop)
5. فرستادن فایل‌ها (push) و وارد شدن به مخزن
6. راستی‌آزمایی: چه چیزی باید دیده شود و چه چیزی **نباید** دیده شود
7. چراغ سبزِ تست‌های خودکار (Actions)
8. دعوت کارفرما و روش‌های نظر دادن
9. آپدیت بعدی، خطاهای رایج و چک‌لیست امنیتی

سه اصطلاح، به زبان ساده

مخزن (Repository): یک پوشهٔ اشتراکی روی اینترنت که هر نسخهٔ فایل‌ها را نگه می‌دارد؛ مثل یک بایگانی نسخه‌بندی‌شده که هر کسی با لینک می‌تواند ببیند.

push: فرستادن کپی نهایی پوشهٔ خودت به آن بایگانی. تا push نکنی، هیچ‌کس چیزی نمی‌بیند.

Token (کلید دسترسی): رمزِ موقت مخصوص همین کار و همین دستگاه — به‌جای رمز اصلی حساب. اگر گم شد، با یک کلیک باطل می‌شود و رمز اصلی حساب دست‌نخورده می‌ماند.

کاری که من (Qoder) قبلاً انجام داده‌ام و کاری که فقط تو می‌توانی بکنی

آماده است: README با اسکرین‌شات‌های واقعی، `LICENSE` (مالکیت خصوصی، فقط بازبینی)، `CONTRIBUTING.md`، `docs/ARCHITECTURE.md`، `CHANGELOG.md`، `SECURITY.md`، قالب‌های Issue و PR، فایل Workflow تست خودکار، `.gitignore` و `.gitattributes`. همه روی همین ماشین، در `Desktop\nobat` (روی دسکتاپ همین ماشین).

فقط تو: ساخت حساب گیت‌هاب، ساخت مخزن، Token، زدن رمز، و دعوت کارفرما. هیچ‌کدام از اینها را نمی‌توانم از طرف تو انجام بدهم — چون به ایمیل و گوشی خودت نیاز دارند.

۱. پیش‌نیازها — قبل از هر کلیک

- [ ] یک **ایمیل فعال** که می‌توانی همین حالا باز کنی (برای تأیید گیت‌هاب).
- [ ] یک **اپلیکیشن تأیید دومرحله‌ای** روی گوشی: Google Authenticator، Microsoft Authenticator یا Aegis. (پیامک هم کار می‌کند ولی ضعیف‌تر است و هزینه دارد.)
- [ ] مرورگر **Chrome** (روی این ماشین هست) و اینترنت پایدار.
- [ ] **Git** نصب است — نسخهٔ `2.55.0.windows.3`؛ چک کردیم.
- [ ] **Python** روی PATH هست.
- [ ] گیت‌هاب CLI (`gh`) **نصب نیست**. اشکالی ندارد — مرحلهٔ ۴ هم با خط فرمان و هم با راه گرافیکی توضیح داده شده.

> **چرا «تأیید دومرحله‌ای» لازم است؟**
> گیت‌هاب از ۲۰۲۳ برای همهٔ حساب‌ها 2FA را اجباری کرده. بدون آن، موقع ساخت مخزن یا Token گیر می‌کنی و پیام «Two-factor authentication is required» می‌بینی. پس مرحلهٔ ۲ را رد نکن.

۲. حساب گیت‌هاب + تأیید دومرحله‌ای

۲-الف) ساخت حساب (اگر نداری)

1. در Chrome به نشانی `github.com` برو. بالا-راست روی **Sign up**.
2. ایمیل را بنویس → **Continue** → یک رمز قوی (حداقل ۱۵ کاراکتر) → **Continue** → نام‌کاربری (username) انتخاب کن.
   این نام در همهٔ لینک‌ها دیده می‌شود، پس رسمی انتخاب کن؛ مثلاً `ali-rezaei`. فارسی و فاصله مجاز نیست.
3. تیک «I'm not a robot» → **Create account** → به ایمیلت یک کد ۶ رقمی می‌آید؛ آن را وارد کن (**Verify**).

۲-ب) روشن کردن 2FA (اگر حساب داری و خاموش است)

1. نشانی کوتاه: `github.com/settings/security` را باز کن. (یا: آواتار بالا-راست ← **Settings** ← منوی چپ ← **Password and authentication**.)
2. در بخش **Two-factor authentication** روی **Set up** یا **Enable two-factor authentication** کلیک کن. اگر رمز حسابت را خواست، همان رمز ورود را بزن.
3. گزینه **Use authentication app** (اپلیکیشن تأیید) را انتخاب کن — نه SMS.
4. یک **QR code** نشان می‌دهد. در گوشی: اپ Authenticator را باز کن ← **Add account** / علامت «+» ← **Scan a QR code** ← دوربین را مقابل مانیتور بگیر. اگر دوربین نداشتی، متن زیر **Secret key** (`Secret key`) را دستی وارد کن.
5. کد ۶ رقمی که اپ نشان می‌دهد را در صفحهٔ گیت‌هاب بنویس (هر ۳۰ ثانیه عوض می‌شود) ← **Verify**.
6. صفحه‌ای با **۱۶ کد بازیابی (Recovery codes)** می‌آید. روی **Download or copy them** بزن و در یک جای امن بگذار (فایل در پوشهٔ Documents، یا چاپ روی کاغذ). **این تنها راهی است اگر گوشی را گم کنی.**

> **هشدار**
>
> کدهای بازیابی و Token را **هرگز** داخل پوشهٔ پروژه (`Desktop\nobat`) ذخیره نکن — همان پوشه‌ای که قرار است روی گیت‌هاب برود. آنها را بیرون از این پوشه بگذار: پوشهٔ `Documents` خودت، یا Password Manager مرورگر.

۳. ساخت مخزن به نام `nobatin`

1. به نشانی `github.com/new` برو. (یا: علامت «+» بالا-راست ← **New repository**.)
2. **Owner:** همان نام‌کاربری خودت. اگر عضو سازمان/گروهی، در این مرحله *حساب شخصی* خودت را انتخاب کن — نه نام سازمان.
3. **Repository name:** دقیقاً بنویس `nobatin` — کوچک، بدون فاصله، بدون فارسی. اگر زیرش قرمز نوشت *"name already exists"*، چیزی مثل `nobatin-app` یا `nobatin-v1` بزن.
4. **Description:** این یک خط را کپی کن:

```
نوبتین — سامانهٔ نوبت‌دهی آنلاین (Flask + SQLite + Jinja). نسخهٔ بازبینی ۱.۰.
```

5. **Visibility:** دو دکمه رادیویی دارد.

گزینه	یعنی چه	برای ما
`Public`	هر کسی در اینترنت لینک را باز کند و کد را ببیند. کارفرما بدون هیچ کاری می‌بیند.	توصیهٔ تو
`Private`	فقط تو و کسایی که دعوت کرده‌ای. کارفرما باید «Collaborator» شود.	اگر حساس است

تصور ساده: Public یعنی ویترین مغازه؛ Private یعنی دفتر پشت مغازه. چون `LICENSE` این مخزن «مالکیت خصوصی، فقط بازبینی» است، Public بودن به کسی حق استفادهٔ تجاری نمی‌دهد — ولی کد قابل دیدن است.

6. **Initialize this repository with a README** → **تیک را بردار**. این مهم‌ترین قدم است: اگر تیک داشته باشد، گیت‌هاب یک README خالی می‌سازد و push اولِ تو با خطا رد می‌شود (خطای شمارهٔ ۳ در بخش ۹). **README واقعی را خودمان داریم.**
7. **Add .gitignore:** منو را روی `No .gitignore` بگذار (ما خودمان `.gitignore` داریم). **Add license:** روی `No license` (فایل `LICENSE` اختصاصی خودمان را داریم).
8. پایین صفحه **Create repository**.

حالا یک صفحهٔ سفید با تیتر *Quick setup — if you've done this kind of thing before* و نشانی `https://github.com/<نام‌کاربری>/nobatin.git` می‌بینی. **این یعنی مخزن خالی آماده است.** آن نشانی را برای مرحلهٔ ۵ کپی کن.

۴. ساخت Token (کلید فرستادن فایل)

گیت‌هاب از مه ۲۰۲۱ دیگر با «رمز حساب» اجازهٔ push نمی‌دهد. باید یک Token بگیری.

1. نشانی: **Generate new token** ← github.com/settings/tokens ← زیرمجموعهٔ **Generate new token (classic)**. رمز حساب + کد ۲FA را می‌خواهد.
2. **Note (نام کلید):** `nobatin-push` — تا بعداً بدون شک بتوانی باطلش کنی.
3. **Expiration:** `30 days` (پیش‌فرض `7 days` را عوض کن؛ ۹۰ روز یا No expiry توصیه نمی‌شود).
4. **Select scopes:** فقط تیک اول — `repo` (Create, read ... private repositories). بقیه را نزن.

```
[x] repo          ← فقط همین یکی
[ ] workflow    [ ] admin:org    [ ] user    [ ] delete_repo    [ ] gist    ← بقیه خاموش
```

5. پایین صفحه **Generate token** ← یک رشته با `ghp_` شروع می‌شود. آیکون **Copy** (کلیپبورد) کنارش را بزن تا کپی شود.

> **Token فقط یک بار نشان داده می‌شود**
>
> اگر جایش نذاشتی، «Generate token» دوباره بساز و قبلی را **Delete** کن. Token قابل بازیابی نیست و قابل دیدن دوباره هم نیست.

> **اگر گزینهٔ «classic» را ندیدی**
>
> گیت‌هاب دارد *Fine-grained tokens* را جا می‌اندازد. در آن حالت: **Generate new token** → `Repository access: All` → `repositories` → `Contents: Read and write` → بقیه `No access` → Generate. خروجی با `github_pat_` شروع می‌شود و همان‌جور استفاده می‌شود.

۵. فرستادن فایل‌ها — دو راه

> **راه الف را انتخاب کن اگر با صفحهٔ سیاهِ دستورات راحتی؛ راه ب را اگر نه.**
>
> هیچ‌کدام امنیتی‌تر نیستند. هر دو همان یک مخزن را پر می‌کنند.

۵-الف) خط فرمان (Git Bash) — توصیه‌شده برای همین پروژه

1. در منوی Start تایپ کن `Git Bash` و باز کن. یک پنجرهٔ سیاه می‌آید. (پاست کردن در این پنجره با **کلیک راست** یا `Ctrl+Shift+V` است، نه `Ctrl+V`.)

2 برو داخل پوشهٔ پروژه:

```
cd ~/Desktop/nobat
```

در Git Bash علامت `~` یعنی پوشهٔ خانگیِ ویندوز (درایو C ← پوشهٔ Users ← نام‌کاربریِ تو ← Desktop)، پس این دستور روی هر ماشین دیگری هم کار می‌کند. اگر پوشهٔ پروژه را جای دیگری برده‌ای، مسیر کاملش را بنویس.

3 **معرفی خودت** (این نام و ایمیل پای هر کامیت می‌نشیند و در گیت‌هاب قابل دیدن است). پیشنهاد: ایمیلِ حریم‌خصوصیِ گیت‌هاب تا ایمیل واقعی‌ت لو نرود — از github.com/settings/emails خطی که با `users.noreply.github.com` تمام می‌شود را بردار:

```
# نام و ایمیل خودت را داخل گیومشن‌ها بگذار
git config --global user.name "Ali Rezaei"
git config --global user.email "12345678+alirezaei@users.noreply.github.com"

# چک کن درست نشسته:
git config --global user.name
git config --global user.email
```

اگر ترجیح می‌دهی تنظیمات سراسری دست‌نخورده بماند، همین مقادیر را با `-c` پای خودِ کامیت بگذار؛ بخش «چند کامیت منطقی» پایین همین صفحه.

4 **ساخت مخزن محلی** (فقط یک بار، و اگر من قبلاً این کار را نکرده‌ام):

```
git init
```

5 **افزودن فایل‌ها** و `دیدن قبل از فرستادن`:

```
git add -A
git status --short | head -30
git ls-files | wc -l        # باید ۹۷ باشد، نه چند هزار
```

6 **تضمین کن هیچ چیز خصوصی داخلش نیست** — این قدم را رد نکن:

```
git ls-files | grep -E '(^|/)instance/|(^|/)\.env($|[.-])|\.(db|sqlite|sqlite3|pem|key|log)$'
\
  | grep -vE '(^|/)\.env\.example$' \
  || echo "پاک است — هیچ فایل خصوصی ردگیری نشده"
```

خروجیِ درست: `پاک است — هیچ فایل خصوصی ردگیری نشده`. اگر لیستی چاپ شد، بخش ۹-ح را بخوان و **قبل از push** اصلاح کن. (دقت: `.env.example` و `deploy/nobatin.env.template` باید در مخزن باشند؛ الگو فقط فایل‌های واقعیِ `.env`، دیتابیس‌ها، لاگ‌ها و کلیدها را می‌گیرد — همان کاری که شغل «بهداشت مخزن» در CI می‌کند.)

7 **کامیت** (ثبت نسخه):

```
git commit -m "سامانهٔ نوبت‌دهی آنلاین — نوبتین ۱.۰ (Flask + SQLite)"
```

8. **وصل کردن به گیت‌هاب و فرستادن** — در نشانی زیر `نام‌کاربری` را عوض کن:

```
git branch -M main
git remote add origin https://github.com/نام‌کاربری/nobatin.git
git push -u origin main
```

9. یک پنجرهٔ «Sign in to GitHub» باز می‌شود (Git Credential Manager). گزینهٔ **Sign in with Browser** ← تب مرورگر ← **Authorize git-credentials**. اگر به‌جای پنجره، در همان صفحهٔ سیاه `Username:` و `Password:` خواست: Username = نام‌کاربری گیت‌هاب، **Password = همانهٔ Token (همان `ghp_…`)** — رمز حساب را نزن، رد می‌شود.

10. چند خط `Writing objects: 100%` که رد شد، تمام. برو github.com/نام‌کاربری/nobatin و F5 بزن.

چند کامیت منطقی (به‌جای یک کامیت بزرگ) — همان چیزی که من انجام می‌دهم

اگر نام و ایمیل را به من بدهی، من مخزن را `init` و در ۵ کامیت جدا می‌چینم، **بدون تغییر تنظیمات سراسریِ git**:

```
git -c user.name="..." -c user.email="..." commit -m "core: schema، jalali، booking، csrf، cache"
routes + templates → static → tests/tools → docs + deploy + .github :به همین ترتیب //
```

بعد از آن تو فقط `git remote add` و `git push` را می‌زنی.

۵-ب) راه گرافیکی: GitHub Desktop (بدون دستور)

1. از desktop.github.com نصب کن (حدود ۱۵۰ مگ). موقع نصب حساب گیت‌هاب را با ۲FA وصل می‌کند.

2. منوی بالا: **Add existing repository** ← **Local repository…** ← **File** ← با Browse پوشهٔ `Desktop\nobat` را انتخاب کن ← **Add repository**. (این کار همان `git init` محلی است.)

3. ستون چپ لیست فایل‌های تغییریافته را می‌آورد. تیک همه را روشن بگذار ← پایین همان ستون در **Summary** بنویس `نوبتین ۱.۰` ← **Commit to main**.

4. بالا: Repository name: `nobatin` ← **Publish repository** ← اگر می‌خواهی فقط کارفرما ببیند تیک **Keep this code private** را روشن بگذار ← **Publish repository**.

5. دکمهٔ **Push origin** (بالا-راست). حالا مخزن پر است.

نکته: GitHub Desktop فایل‌های نادیده‌گرفته‌شده (`instance/` و `.env`) را خودش نشان نمی‌دهد، چون `.gitignore` را محترم می‌شمارد. اگر لیستش چیزی شبیه `.env` نشان داد، به بخش ۹-ح برو و push نکن.

۶. راستی‌آزمایی — چه چیزی باید دیده شود

۶-الف) در صفحهٔ اصلی مخزن

باید دیده شود	نباید باشد — و معنیش
`README.md` با تصاویر فارسی، بخش‌های ۱ تا ۸ و بنرهای بالا	`instance/nobatin.db` — دیتابیس واقعی/تست لو رفته
پوشه‌های `static`، `templates`، `routes`، `core`، `.github`، `tools`، `docs`، `deploy`	`.env` — همهٔ رمزها و کلیدها
`SECURITY.md`، `CONTRIBUTING.md`، `LICENSE`، `.gitattributes`، `.gitignore`، `CHANGELOG.md`	`__pycache__/` و `*.log` — شلختگی (خطرناک نیست، ولی نشان می‌دهد .gitignore کار نکرده)
`app.py`، `requirements.txt`، `smoke_test.py`، `wsgi.py`	`nobatin_test.db` — دیتابیس آزمون

۶-ب) جست‌وجوی داخل مخزن (۳۰ ثانیه، ارزشش را دارد)

در صفحهٔ مخزن، کادر جست‌وجوی بالا-چپ را بزن (کلید `/`)، روی همان مخزن قفل کن و این را تایپ کن تا مطمئن شوی رمز جایی نمانده:

```
NOBATIN_SECRET
```

خروجی درست: فقط `.env.example` و `deploy/nobatin.env.template` و `README.md` — و در اینها مقدار پشت علامت `=` **خالی** است. اگر رشته‌ای شبیه `NOBATIN_SECRET=abc123…` دیدی، فوراً بخش ۹-ح.

۶-پ) تنظیمات نمایش مخزن

1. تب **Settings** (چرخ‌دنده، بالای ستون‌های مخزن — نه Settings حساب).
2. در **General** → **Features**: تیک `Issues` **روشن** بماند — جای اصلی نظر کارفرماست. `Wiki` و `Discussions` را می‌توانی خاموش کنی تا شلوغ نشود.
3. در همان صفحه **Repository description** و در **Website** اگر سروری داری نشانی‌اش را بنویس (فعلاً خالی بگذار).
4. پایین‌تر: **Default branch** باید `main` باشد. اگر `master` است، CI باز هم اجرا می‌شود (هر دو را در فایل workflow گذاشته‌ایم).

۷. تست‌های خودکار: چراغ سبز Actions

فایل `.github/workflows/ci.yml` را گذاشته‌ایم. معادل ساده‌اش: «یک ماشین مجازی در اینترنت، هر بار که فایلی می‌فرستی، خودش پروژه را نصب می‌کند و همهٔ آزمون‌ها را اجرا می‌کند و به تو کارتی می‌دهد.»

1. در مخزن، تب بالا **Actions**.

2. اولین اجرا خودش شروع شده (*Workflow runs*). نقطهٔ زرد کنارش یعنی «در حال اجرا». معمولاً **۲ تا ۶ دقیقه**.

3. بازنشانی کن (F5). باید سه کارت سبز ببینی:

شغل	چه کاری می‌کند	زمان تقریبی
آزمون‌ها (Python 3.10 … 3.13)	۴ نسخهٔ پایتون × ۳۵ — `python smoke_test.py` بخش آزمون	۲–۴ دقیقه
اجرای واقعی + production سرصفحه‌ها	`wsgi.py` را با حالت production بالا می‌آورد و HSTS/CSP/CSRF/رد شدن هاست جعلی و `/healthz` را می‌سنجد؛ به‌علاوه دو حالتِ «بالا نیامدن» (بدون رمز و با OTP master)	۴۰–۹۰ ثانیه
بهداشت مخزن	هیچ `.env` /دیتابیس/لاگ ردگیری نشده؛ هیچ مسیر شخصیِ ماشین توسعه (درایو C و پوشهٔ Users) در فایل‌ها نیست؛ هیچ `<script>` درون‌خطی نیست؛ CDN نیست	۱۵–۳۰ ثانیه

4. روی نام اجرا کلیک کن ← روی هر شغل ← **Set up job** / **Summary** برای دیدن لاگ کامل. اگر قرمز شد: لاگ آن «step» قرمز را کپی کن (بخش ۹-د).

5. **چک نهایی برای کارفرما:** در صفحهٔ اصلی مخزن، کنار آخرین کامیت باید نشانهٔ سبز «passing» باشد. اگر نیست، یعنی تست‌ها رد شده‌اند و کارفرما همان را می‌بیند.

اختیاری: افزودن نشان (badge) وضعیت تست به README

بعد از ساخت مخزن، این خط را **بعد از** بنرهای موجودِ README اضافه کن و `OWNER` را با نام‌کاربری خودت عوض کن — اگر عوض نکنی نشان «invalid» می‌شود:

```
[![CI](https://github.com/OWNER/nobatin/actions/workflows/ci.yml/badge.svg)]
(https://github.com/OWNER/nobatin/actions/workflows/ci.yml)
```

۸. دعوت کارفرما و گرفتن نظر

۸-الف) اگر مخزن Public است

کارفرما اصلاً نیاز به دعوت ندارد: لینک را بده، تمام.

```
https://github.com/نام‌کاربری/nobatin
```

برای نظر روی خط مشخص کد نیاز به حساب گیت‌هاب دارد (بدون حساب فقط می‌تواند ببیند). اگر حساب ندارد، مخزن را Private کن و او را دعوت کن (۸-ب).

۸-ب) دعوت رسمی (Collaborator) — لازم برای مخزن Private، و برای Write دادن

1. مخزن → **Settings** → منوی چپ (زیر *Access*) → **Collaborators and teams** → **Add people**.
2. نام‌کاربری یا ایمیل کارفرما را تایپ کن، از لیست انتخابش کن → **Add to repository**.
3. سطح دسترسی: برای «فقط ببیند و نظر بدهد» `Read` کافی است. اگر می‌خواهد خودش شاخه بسازد و PR بفرستد `Triage` یا **Admin** . **نده** `Write` — با Admin می‌تواند مخزن را پاک کند.
4. برای او ایمیل دعوت می‌رود؛ وقتی **Accept invitation** زد در لیست می‌آید.

۸-پ) چهار راهی که کارفرما می‌تواند نظر بدهد

روش	مسیر کلیک	کِی استفاده کند
۱. Issue (ایراد / درخواست)	تب **Issues** → **New issue** → یکی از دو قالب آماده: «گزارش ایراد» یا «درخواست قابلیت» — یا «New issue» سفید (allow empty issues را روشن گذاشته‌ایم)	برای ۹۰٪ نظرها. قالب‌ها خودش می‌پرسند: چه شد، مراحلی که زدی، محیط، پایتون/Flask، لاگ. لاگ را **بدون** شمارهٔ موبایل و رمز بفرستد.
۲. کامنت روی یک خط کد	فایل را باز کن (مثلاً `core/booking.py`) → روی **شمارهٔ خط** سمت چپ کلیک کن → دکمهٔ **...** که ظاهر می‌شود → **Comment** → بنویس → **Update comment**	«اینجا قفل ساعت را درست ندیده‌ام» — دقیق‌ترین نوع بازبینی.
۳. Review روی Pull Request	تب **Pull requests** → یک PR → **Files changed** → **Create review** → `Comment` / `Approve` / `Request changes`	وقتی خودش تغییری فرستاد و تو خواستی او تأیید کند.
۴. گفت‌وگوی ساده	همان Issue با Markdown: برای کد سه تا backtick، برای تیک‌زدنی `- [ ]`	لیست چک نهایی: یک Issue به نام «بازبینی نسخهٔ ۱.۰» بساز، لینک `docs/ARCHITECTURE.md`، `README`، `deploy/README.md` را داخلش بده و از کارفرما بخواه تیک بزند.

یک Issue «درِ ورود» بساز — تا کارفرما گم نشود

1. تب **Blank issue** ← **New issue** ← **Issues**.
2. عنوان: `بازبینی نوبتین ۱.۰ — شروع از اینجا`
3. متن (کپی کن):

```
این مخزن نسخهٔ بازبینی نوبتین ۱.۰ است.

پیشنهاد من برای ترتیب دیدن:
1. README.md — «معرفی، اسکرین‌شات‌ها، «پیاده‌شده ⇔ عمداً شبیه‌سازی‌شده»
2. docs/ARCHITECTURE.md — چهار دیاگرام (توالی رزرو/پرداخت، موجودیت‌ها، نمای کلی، لایهٔ WSGI)
3. تب Actions — سه شغل سبز = ۳۵ بخش آزمون روی پایتون ۳.۱۰ تا ۳.۱۳
4. deploy/README.md — اگر بخواهیم روی سرور ببریم
5. smoke_test.py — خودِ آزمون‌ها

درگاه بانکی و سرویس پیامک عمداً شبیه‌سازی‌اند (بخش ۳-ب README).
نظرت را همین‌جا یا روی خط کدها بنویس.
```

4. **Submit new issue** ← لینکش را برای کارفرما بفرست (آیکون «Copy link» در کامنت‌ها، یا نشانی نوار مرورگر).

۹. بعد از نظرها: آپدیت، و رفع خطاها

۹-الف) فرستادن تغییرات بعدی (هر تعداد که باشد)

```
cd ~/Desktop/nobat
git status                          # چه چیزی عوض شده؟
python smoke_test.py                # قبل از فرستادن، آزمون محلی
git add -A
git commit -m "رفع نظر کارفرما: پیش‌نمایش بازپرداخت در لغو"
git push
```

هر push = یک اجرای جدید Actions. اگر کارفرما روی موبایل باشد، نتیجهٔ سبز/قرمز را همان‌جا در Issue می‌بیند.

۹-ب) اگر کارفرما خودش changes فرستاد (PR از Fork)

```
git fetch origin pull/7/head:review-7
git checkout review-7
python smoke_test.py
# اگر خوب بود برگرد:
git checkout main
git merge review-7
git push
```

عدد ۷ شمارهٔ PR است و در آدرس صفحهٔ PR نوشته شده. اگر `gh` نصب کنی، همان‌ها یک دستور می‌شود: `gh pr checkout 7`.

۹-پ) هشت خطای رایج — پیام واقعی و راه‌حل

چیزی که می‌بینی	چرا	راه‌حل
`remote: Support for password authentication was removed` و `Authentication failed`	جای Token، رمز حساب را زده‌ای	Username = نام‌کاربری، Password = همان `ghp_…`. اگر قبلاً اشتباه ذخیره شده: Control Panel → *Credential Manager* → Windows Credentials → `git:https://github.com` → Remove → دوباره push.
`403 Forbidden` / `Permission denied` / `Invalid token`	Token scope ندارد (`repo` تیک نبود) یا منقضی شده یا با نام کاربری اشتباه فرستادی	در `github.com/settings/tokens` ببین منقضی نشده باشد؛ Token تازه با scope `repo` بساز؛ مخزن متعلق به آن حساب است؟
`! [rejected] main -> main (fetch first)` یا «the remote contains work you do not have locally»	موقع ساخت مخزن تیک «Add a README» روشن مانده بود؛ مخزن پس از سمت گیت‌هاب یک کامیت اولیه دارد و تاریخچه‌های محلی و ریموت به هم نمی‌خورند	**راه معمولی:** `git pull --rebase origin main` ← اگر وسط کار روی `README.md` اختلاف آمد، نسخهٔ خودت را نگه‌دار: `git checkout --ours README.md && git add README.md && git rebase --continue` ← بعد `git push -u origin main`. **راه پاک (مخزن هنوز خالی است و داده‌ای از دست نمی‌رود):** در مخزن **Settings** → پایین‌ترین بخش قرمز **Delete this repository** → دوباره بساز و مطمئن شو تیک README **خاموش** است.
`fatal: unable to auto-detect email address` / «Please tell me who you are»	قدم ۳ از ۵-الف (معرفی نام و ایمیل) را نزدی	همان دو خط `git config --global …` را بزن و دوباره `git commit`.
push خیلی کند است یا `RPC failed; curl 56`	معمولاً یک فایل بزرگ ناخواسته (دیتابیس/لاگ) رفته داخل کامیت	`git ls-files | wc -l` (باید ۹۷) و `du -sh .git` چک شود. اگر بزرگ بود: فایل را از index در بیاور (۹-ح) و کامیت‌ها را از اول بساز.
Actions قرمز در شغل «آزمون‌ها»	نسخهٔ پایتون، یا خطای واقعی در تست	روی شغل قرمز ← step قرمز ← لاگ. اگر `RuntimeError: production …` بود یعنی یکی از گاردهای fail-closed فعال شده — که خودش خبر خوب است؛ لاگ را در یک Issue بگذار و برایم بفرست.
Actions قرمز در «بهداشت مخزن» با لیست فایل	یک فایل خصوصی یا مسیر شخصیِ ماشین توسعه وارد مخزن شده	۹-ح — قبل از آن، لینک مخزن را برای کسی نفرست.
فارسی‌ها در صفحهٔ گیت‌هاب به‌هم‌ریخته یا «؟» است	فایل با encoding اشتباه ذخیره شده	همهٔ فایل‌های این پروژه UTF-8 و با `.gitattributes` خط‌پایان LF هستند. اگر در ویرایشگر خودت فایل را دوباره ذخیره کردی، UTF-8 را انتخاب کن.

۹-ح) اگر فایل خصوصی (مثل `.env` یا `*.db`) رفت داخل مخزن

ترتیب درست، به همین سرعت:

1. **اگر رمز واقعی داخلش بود، اول باطلش کن.** Token را در `github.com/settings/tokens` با **Delete** پاک کن، `NOBATIN_SECRET` را با یک رشتهٔ تازه عوض کن، کلید پیامک را در پنل سرویس‌دهنده rotate کن. رازِ لورفته روی مخزن Public **چند ثانیه** بعد توسط ربات‌ها پیدا می‌شود؛ حذف فایل، رمز را بی‌اثر نمی‌کند.

2. فایل را از مخزن دربیاور ولی از دیسک پاک نکن:

```
git rm --cached .env          # یا instance/nobatin.db
echo ".env" >> .gitignore     # اگر خطش نبود
git add .gitignore
git commit -m "حذف فایل خصوصی از مخزن + تکمیل .gitignore"
git push
```

3. **تاریخچه هنوز فایل را دارد.** اگر واقعاً رازی در تاریخچه بوده: Delete commit history» → `github.com/contact` » درخواست بده، یا مخزن را پاک کن و دوباره بساز (این راهنما را از اول اجرا کن). ساده‌تر و مطمئن‌تر: قبل از push اول، چکِ قدم ۶ از ۵-الف را جدی بگیر.

4. راستی‌آزمایی نهایی (دقیقاً همان چیزی که CI هم می‌سنجد):

```
git ls-files | grep -E '(^|/)instance/|(^|/)\.env($|[.-])|\.(db|sqlite|sqlite3|pem|key|log)$'
\
  | grep -vE '(^|/)\.env\.example$' \
  || echo "پاک است"
```

۱۰. چک‌لیست «آماده برای کارفرما»

- [ ] ۲FA روی حساب روشن است و کدهای بازیابی جای دیگری ذخیره شده‌اند.
- [ ] Token فقط `repo` scope دارد، تاریخ انقضا (۳۰ روز) دارد و نامش `nobatin-push` است.
- [ ] `git ls-files | wc -l` = ۹۷ و خروجیِ چکِ فایل‌های خصوصی «پاک است» می‌نویسد.
- [ ] در صفحهٔ مخزن نه `instance/` هست، نه `.env`، نه `__pycache__`، نه `*.db`.
- [ ] جست‌وجوی `NOBATIN_SECRET` در مخزن فقط مقدار **خالی** در `.env.example` و `deploy/nobatin.env.template` را نشان می‌دهد.
- [ ] تب Actions سه شغل سبز دارد و کنار آخرین کامیت نشان «passing» سبز است.
- [ ] README: بخش «پیاده‌شده ↔ عمداً شبیه‌سازی‌شده» (درگاه `sandbox`، پیامک `log`) و «چیزی که خواسته شده بود ↔ جایی که پیاده شده» هر دو دیده می‌شوند.
- [ ] `LICENSE` در صفحهٔ مخزن «Other / proprietary» است — نه MIT/Apache. یعنی بازبینی آزاد، استفاده و استقرار نه.
- [ ] دادهٔ نمونه فقط شماره‌های ساختگی است (`09120000000` و همسان‌ها) و شمارهٔ واقعی هیچ‌کس در seed نیست.
- [ ] یک Issue «بازبینی نسخهٔ ۱.۰ — شروع از اینجا» ساخته شده و لینکش برای کارفرما رفته.
- [ ] برای مخزن Private: دعوت Accept شده و سطحش `Read`/`Triage` است (`Admin` نه).

۱۰-ب) سه چیزی که کارفرما سؤال می‌کند — جواب حاضر

سؤال	جواب کوتاه
«چرا درگاه پول نمی‌گیرد؟»	عمداً `core/gateway.py` صفحهٔ شبیه‌سازی است؛ برای درگاه واقعی فقط همان یک فایل و صفحهٔ بازگشت عوض می‌شوند. جدول تراکنش‌ها، وضعیت‌ها و قاعدهٔ «پرداخت ناموفق = نوبت قطعی نه» دست‌نخورده می‌ماند. (README بخش ۳-ب و ۶)
«چرا پیامک نمی‌آید؟»	پیش‌فرض فرستندهٔ `log` است تا شماره و هزینه خرج نشود؛ همه در جدول `sms_log` و در صندوق sms-box/ خوانده می‌شوند. برای کاوه‌نگار سه متغیر محیطی کافی است. (README بخش ۴)
«چند کاربر هم‌زمان تحمل می‌کند؟»	روی همین ماشینِ توسعه: ~۱۴۰–۱۶۵ درخواست بر ثانیه برای صفحه‌های عمومی و ~۹۰–۹۸ برای پنل واردشده، بدون ۵xx. یک ساعت را دو نفر هم‌زمان نمی‌توانند بگیرند (قفل تراکنشی + ایندکس یکتای مشروط — شغل ۳۳ آزمون). (README بخش ۲-ب)

این فایل (`docs/github-guide.html`) و خروجی PDFاش در مخزن است. تاریخ تهیه: ۱۱ آبان ۱۴۰۵ (۲ نوامبر ۲۰۲۶). عددها روی ماشینِ توسعه با Git for Windows 2.55.0 و Python 3.14 اندازه‌گیری/چک شده‌اند؛ نام منوها بر پایهٔ رابط انگلیسیِ گیت‌هاب نوشته شده و اگر گیت‌هاب چیزی را جابه‌جا کرد، نشانی‌های مستقیمِ داخل راهنما (مثل `github.com/new` و `github.com/settings/tokens`) کار می‌کنند. نوبتین نسخهٔ ۱.۰ — مالکیت خصوصی، فقط برای بازبینی.